



TECHFLOW SOLUTIONS

Engenharia de Software

Sumário

Descrição do projeto e seu escopo inicial	2
Sistema de Gerenciamento de Tarefas (Kanban).....	2
Funcionalidades principais.....	2
Estrutura técnica	2
Organização do projeto.....	2
Arquitetura.....	3
Resumido	3
Projeto ao fim do desenvolvimento	4
Objetivo	4
Funcionalidades do Sistema.....	4
Arquitetura do Sistema	4
Controle de Status	4
Prioridade das Tarefas	4
Persistência de Dados	4
Interface Web	4
Controle de Projeto	4
Fora do Escopo.....	5
Mudança de Escopo	5
Resumido	5
Metodologia utilizada	5
Kanban.....	5
Scrum	6
Desenvolvimento Incremental.....	6
Gestão de Mudanças	6
Controle de Qualidade	6
Controle de Versão	7
Importância da Modelagem na Engenharia de Software.....	7
Importância no Desenvolvimento.....	7
Entender o Problema	7
Planejar a Estrutura do Sistema	7
Reduzir Falhas	7
Facilitar Manutenção	7
Melhorar Comunicação.....	8
Aplicação no Projeto TaskFlow	8
Benefícios Obtidos	8
Conclusão	8
Diagrama de Casos de Uso	9

Diagrama de Classes.....	10
Classes principais do TaskFlow	10
Classe Usuário	10
Classe Tarefa	10
Classe Status	10
Classe Prioridade.....	10
Relacionamentos	11
Testes automatizados utilizados	12
Objetivo do teste	12
Funcionamento	12
Importância do teste automatizado.....	12
Prints: Workflow de CI funcionando	12
Principais commits.....	13
Quadro Kanban do TaskFlow	14
Conclusão.....	14

Descrição do projeto e seu escopo inicial

Sistema de Gerenciamento de Tarefas (Kanban)

O objetivo é desenvolver um sistema inspirado em Kanban para gerenciamento de tarefas, aplicando conceitos de Engenharia de Software, GitHub e desenvolvimento Java.

Funcionalidades principais

- Cadastro de usuários
- Gerenciamento de tarefas
- Controle de status
- Controle de prioridade

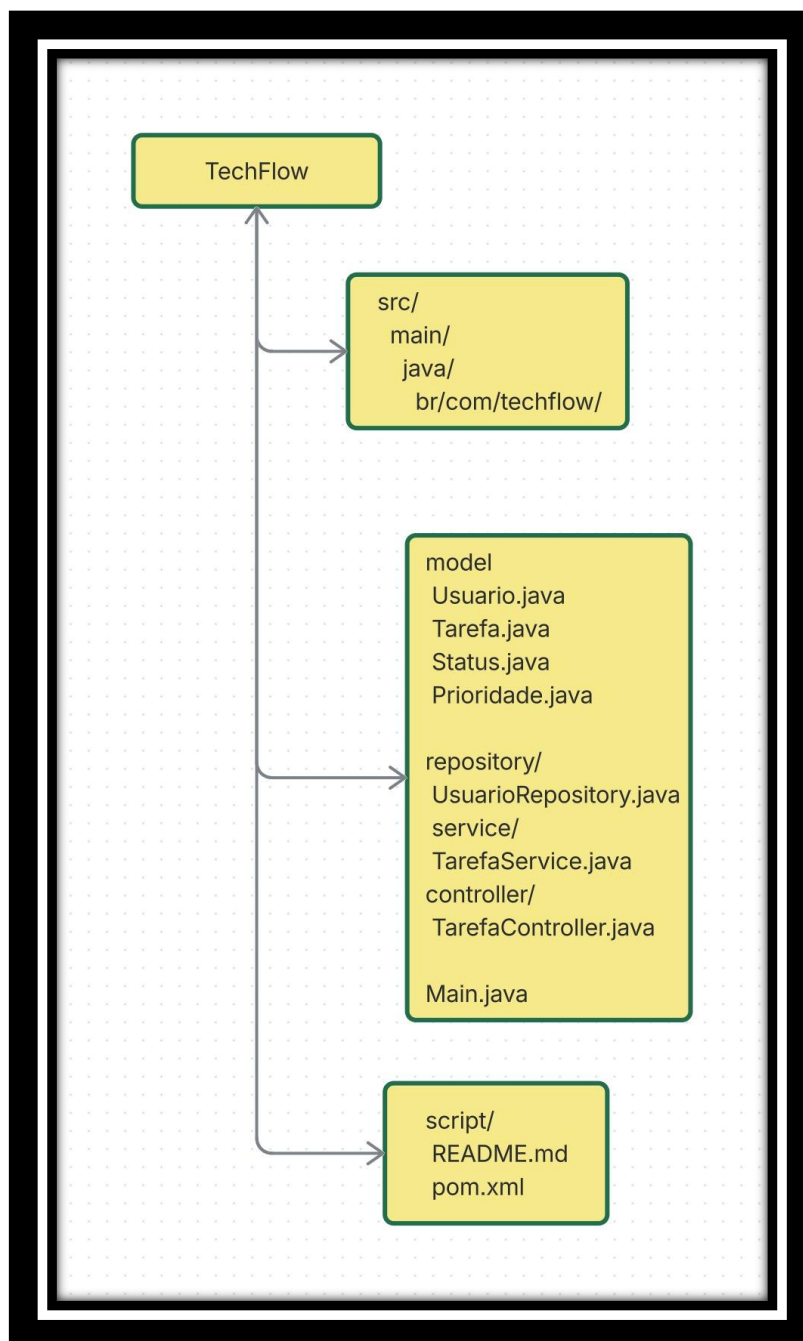
Estrutura técnica

- Java
- Programação Orientada a Objetos
- Encapsulamento
- Construtores
- Packages
- Git/GitHub
- README
- Banco de dados MySQL

Organização do projeto

- estrutura de pastas
- commits organizados
- documentação
- scripts/configuração

Arquitetura



Resumido

Um sistema Java de gerenciamento de tarefas estilo Kanban, com cadastro de usuários, controle de status e prioridade, utilizando orientação a objetos, versionamento com GitHub e futuramente persistência em banco de dados.

Projeto ao fim do desenvolvimento

Objetivo

Desenvolver um sistema web simples para gerenciamento de tarefas utilizando conceitos de metodologias ágeis, permitindo o acompanhamento e organização das atividades de uma equipe.

Funcionalidades do Sistema

- Criar tarefas
- Editar tarefas
- Excluir tarefas
- Listar tarefas

Arquitetura do Sistema

O projeto foi desenvolvido utilizando arquitetura baseada em separação de responsabilidades:

- HTML → estrutura da interface
- CSS → estilização visual
- JavaScript → lógica e funcionalidades

Controle de Status

As tarefas poderão possuir os seguintes status:

A Fazer

- Em Progresso
- Concluído

Prioridade das Tarefas

Cada tarefa poderá possuir:

- Prioridade Baixa
- Prioridade Média
- Prioridade Alta

Persistência de Dados

As informações serão armazenadas utilizando LocalStorage do navegador.

Interface Web

O sistema possuirá:

- Interface simples e responsiva
- Organização visual das tarefas
- Botões de gerenciamento

Controle de Projeto

O desenvolvimento utilizará:

- GitHub para versionamento
- GitHub Projects para Kanban
- GitHub Actions para integração contínua

Fora do Escopo

Para manter o projeto simples e compatível com o prazo acadêmico, não serão implementados:

- Sistema avançado de autenticação
- Banco de dados online
- API REST completa
- Deploy em servidor
- Multiusuários
- Notificações em tempo real

Mudança de Escopo

Durante o desenvolvimento, foi adicionada a funcionalidade de prioridade de tarefas após solicitação simulada do cliente.

A alteração teve como objetivo melhorar a organização das atividades críticas dentro do sistema.

Resumido

Um sistema JavaScript de gerenciamento de tarefas estilo Kanban, com controle de status e prioridade, utilizando versionamento com GitHub e futuramente persistência em banco de dados, as

Metodologia utilizada

O projeto foi desenvolvido utilizando conceitos de metodologias ágeis, com foco principal em Kanban e práticas inspiradas no Scrum.

O objetivo da metodologia adotada foi organizar o fluxo de trabalho, facilitar o acompanhamento das tarefas e permitir adaptação contínua durante o desenvolvimento do sistema.

Kanban

O método Kanban foi utilizado para organizar visualmente as atividades do projeto através do GitHub Projects.

As tarefas foram distribuídas nas seguintes etapas:

- A Fazer
- Em Progresso
- Concluído

Essa abordagem permitiu:

- melhor controle das atividades;
- acompanhamento da evolução do projeto;
- visualização do fluxo de desenvolvimento;
- organização das prioridades.

Scrum

Conceitos do Scrum também foram aplicados no desenvolvimento do projeto, principalmente:

- divisão das tarefas em pequenas entregas;
- desenvolvimento incremental;
- adaptação contínua;
- organização do backlog;
- priorização das funcionalidades.

Desenvolvimento Incremental

O sistema foi desenvolvido em etapas, permitindo evolução gradual das funcionalidades.

Etapas realizadas

1. Planejamento do projeto
2. Criação do repositório
3. Desenvolvimento da interface
4. Implementação do CRUD
5. Adição do sistema de status
6. Implementação de prioridades
7. Configuração do GitHub Actions
8. Atualização da documentação

Gestão de Mudanças

Durante o desenvolvimento, foi simulada uma alteração de escopo solicitada pelo cliente.

Mudança implementada

Adição da funcionalidade de prioridade das tarefas.

Essa alteração exigiu:

- atualização da documentação;
- reorganização do Kanban;
- novas tarefas no backlog;
- adaptação do sistema.

A metodologia ágil permitiu que essa mudança fosse incorporada sem comprometer o andamento do projeto.

Controle de Qualidade

Foi utilizado GitHub Actions para simular integração contínua e automação de verificações do sistema.

Essa prática contribuiu para:

- maior confiabilidade;
- automatização de processos;
- melhor controle das alterações;
- identificação rápida de problemas.

Controle de Versão

O Git e o [GitHub](#) foram utilizados para:

- versionamento do código;
- registro do histórico de desenvolvimento;
- organização das alterações;
- documentação da evolução do projeto.

Os commits foram realizados de forma descritiva e organizada, seguindo boas práticas de desenvolvimento.

Importância da Modelagem na Engenharia de Software

A modelagem é uma das etapas mais importantes da Engenharia de Software, pois permite representar e planejar o sistema antes do desenvolvimento do código.

Ela funciona como um “mapa” do projeto, ajudando a equipe a entender:

- como o sistema irá funcionar;
- quais serão suas funcionalidades;
- como os dados serão organizados;
- e como os componentes irão se comunicar.

Importância no Desenvolvimento

Durante o desenvolvimento de software, a modelagem ajuda a:

Entender o Problema

Permite analisar as necessidades do cliente antes da implementação.

Planejar a Estrutura do Sistema

Define:

- entidades;
- funcionalidades;
- relacionamentos;
- fluxo do sistema.

Reduzir Falhas

Erros podem ser identificados ainda na fase de planejamento, evitando retrabalho.

Facilitar Manutenção

Um sistema modelado corretamente é mais fácil de:

- atualizar;
- corrigir;
- expandir futuramente.

Melhorar Comunicação

A modelagem facilita a comunicação entre:

- desenvolvedores;
- analistas;
- gestores;
- clientes.

Aplicação no Projeto TaskFlow

No projeto TaskFlow, a modelagem foi importante para definir:

- estrutura das tarefas;
- atributos das atividades;
- fluxo do CRUD;
- organização do sistema Kanban;
- controle de status;
- prioridades das tarefas.

Benefícios Obtidos

A modelagem permitiu:

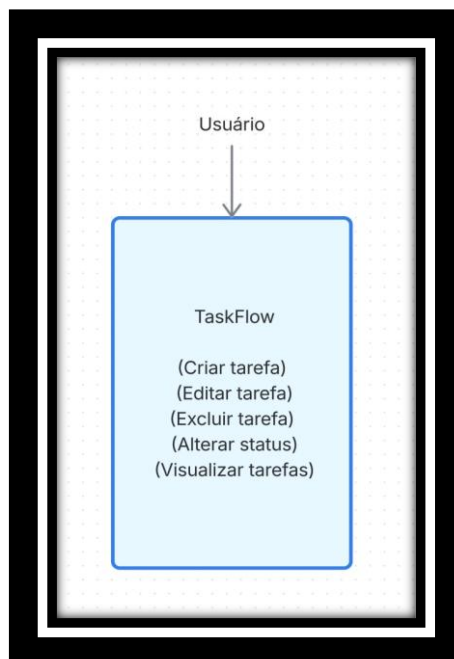
- maior organização do projeto;
- facilidade no desenvolvimento;
- melhor documentação;
- redução de erros;
- controle mais eficiente das funcionalidades.

Conclusão

A modelagem é fundamental na Engenharia de Software porque permite o planejamento do sistema antes da implementação, reduzindo riscos e aumentando a qualidade do software.

Além disso, ela melhora a organização, facilita o desenvolvimento do sistema contribui para a estruturado e eficiência do projeto.

Diagrama de Casos de Uso



O diagrama de casos de uso foi utilizado para representar as interações entre o usuário e o sistema, identificando as principais funcionalidades disponíveis no TaskFlow.

O usuário pode criar e modificar suas tarefas da forma que desejar e alterar os status das suas tarefas podendo especificar se tal tarefa tem baixa, média ou alta prioridade.

Diagrama de Classes

Classes principais do TaskFlow

- Usuario
- Tarefa
- Status
- Prioridade

Classe Usuário

- id : int
- nome : String
- email : String

- + criarTarefa()
- + editarTarefa()
- + excluirTarefa()

Classe Tarefa

- id : int
- titulo : String
- descricao : String
- status : Status
- prioridade : Prioridade

- + alterarStatus()
- + editarTarefa()

Classe Status

- nome : String

Exemplos:

- A Fazer
- Em Progresso
- Concluído

Classe Prioridade

- nivel : String

Exemplos:

- Baixa
- Média
- Alta

Relacionamentos

Usuario → Tarefa

Um usuário pode ter várias tarefas.

Usuario 1 ----- * Tarefa

Tarefa → Status

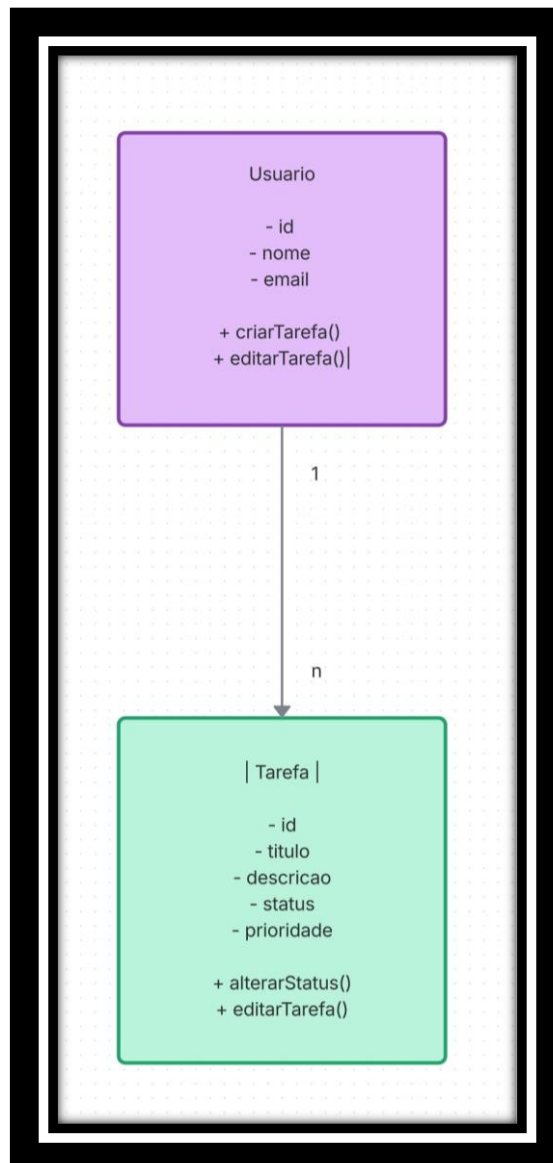
Tarefa ----- 1 Status

Cada tarefa possui um status.

Tarefa → Prioridade

Tarefa ----- 1 Prioridade

Cada tarefa possui uma prioridade.



O diagrama de classes foi utilizado para representar a estrutura do sistema, identificando as principais entidades, seus atributos, métodos e relacionamentos.

Testes automatizados utilizados

Foi configurado um pipeline básico de integração contínua utilizando GitHub Actions para executar verificações automáticas sempre que alterações fossem enviadas ao repositório.

Objetivo do teste

- validar o funcionamento do pipeline;
- simular integração contínua;
- garantir maior controle de qualidade;
- automatizar verificações do projeto.

Funcionamento

Sempre que um push é realizado na branch principal (main), o GitHub Actions executa automaticamente o workflow configurado no arquivo:

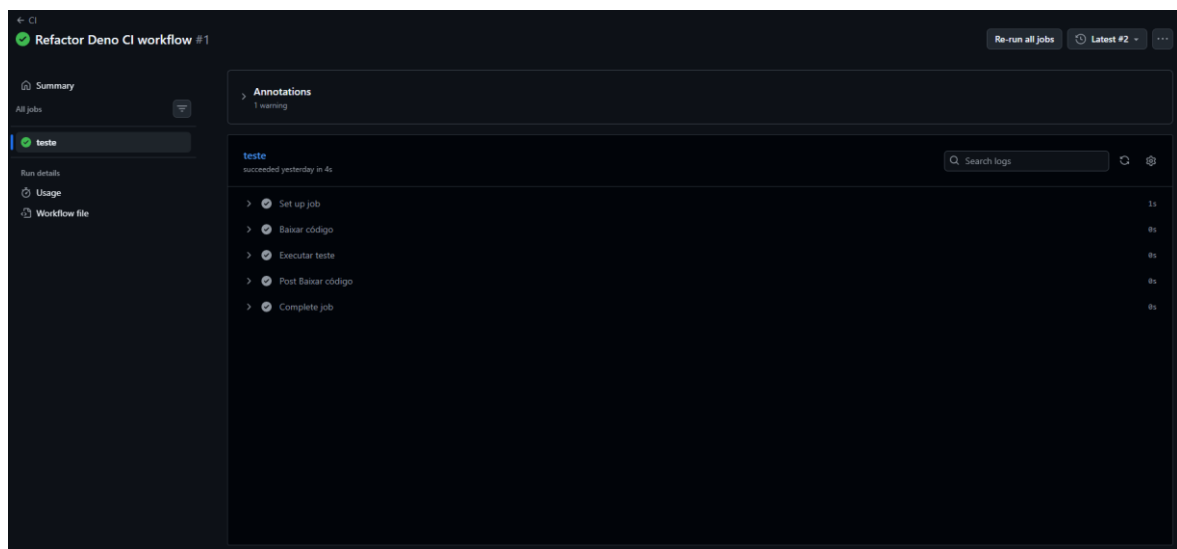
Importância do teste automatizado

A utilização do GitHub Actions permitiu:

- automatizar processos;
- validar alterações automaticamente;
- melhorar a confiabilidade do sistema;
- aplicar conceitos de integração contínua.

O projeto utilizou GitHub Actions para realizar testes automatizados básicos e simular um processo de integração contínua, garantindo maior controle de qualidade durante o desenvolvimento.

Prints: Workflow de CI funcionando



← CI

Refactor Deno CI workflow #1

Re-run all jobs Latest #2

Summary

All jobs

teste

Run details

Usage

Workflow file

Re-run triggered yesterday

RuanPerSouza → d450b3c main

Status Success

Total duration 10s

Artifacts -

deno.yml

on push

teste

Annotations

1 warning

teste

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4. Actions will be forced to run with Node.js 24 by default starting June 2nd, 2025.

Show more

← CI

Refactor Deno CI workflow #1

Re-run all jobs Latest #2

Summary

All jobs

teste

Run details

Usage

Workflow file

Workflow file for this run

github/workflows/deno.yml@d450b3c

```

1 name: CI
2
3 on:
4   push:
5     branches:
6       - main
7
8 jobs:
9   teste:
10    runs-on: ubuntu-latest
11
12    steps:
13
14      - name: Baixar código
15        uses: actions/checkout@v4
16
17      - name: Executar teste
18        run: echo "Sistema testado com sucesso!"

```

Principais commits

Reformulando REDAME com detalhes sobre o projeto e desenvolvimento

Updated project details and structure in README.

Verified 69061d6

RuanPerSouza authored yesterday

Create style.css

Criando o estilo do front

4c0dae

RuanPerSouza committed yesterday

Refactor Deno CI workflow

Updated the Deno CI workflow to simplify steps and change job names.

Verified d450b3c

RuanPerSouza authored yesterday · 1 / 1

Update README.md

Reformulando o README para se encaixar melhor na versão atual do projeto

4ff8e9a

RuanPerSouza committed yesterday

Create ci.yml

implementação que testa o sistema toda vez que ele é enviado para o repositório

9bbe0f5

RuanPerSouza committed yesterday

Create script.js

Criando o backend

c04aefc

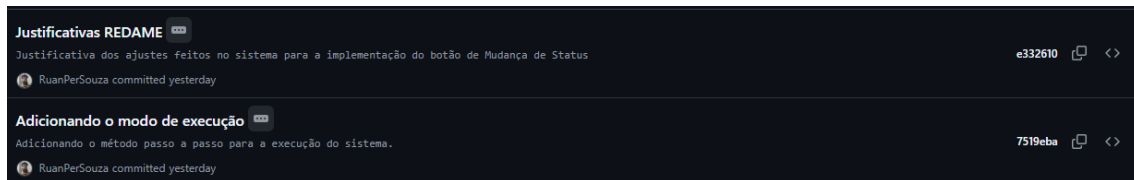
RuanPerSouza committed yesterday

Organização do projeto

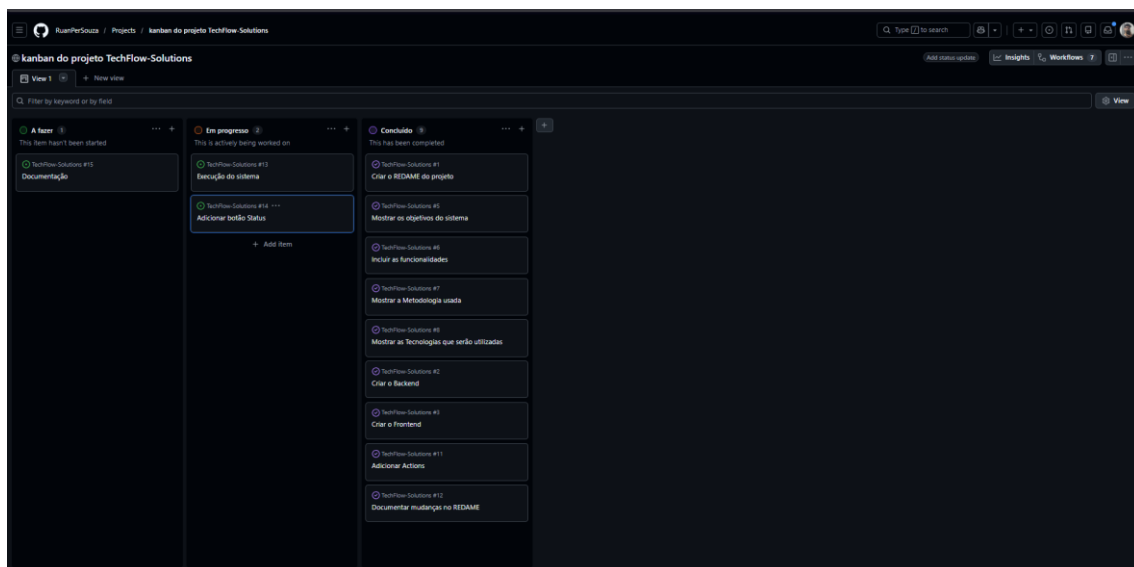
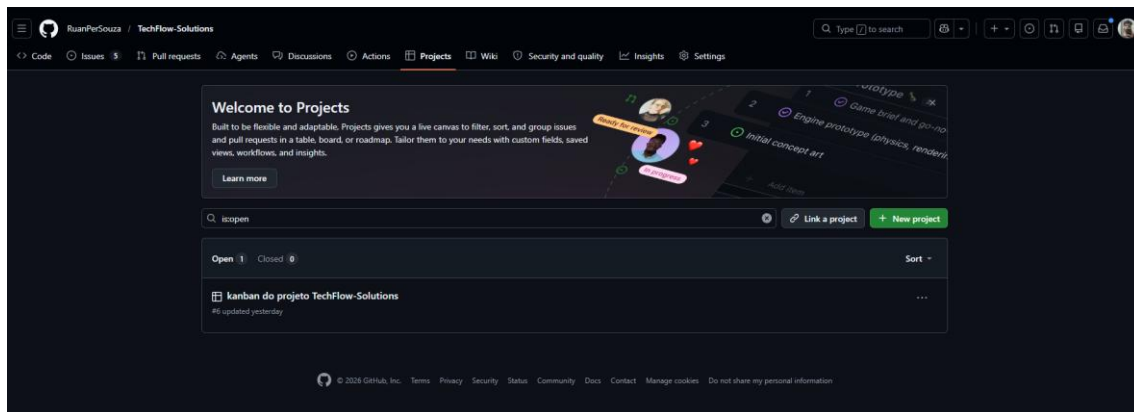
Criando pastas e separando códigos

7175af9

RuanPerSouza committed yesterday



Quadro Kanban do TaskFlow



Conclusão

O desenvolvimento do projeto TaskFlow permitiu aplicar na prática diversos conceitos da Engenharia de Software, como metodologias ágeis, modelagem de sistemas, versionamento de código, integração contínua e controle de qualidade.

Durante o projeto, foi possível compreender a importância da organização das tarefas, da documentação e da adaptação às mudanças de escopo, simulando situações presentes no desenvolvimento real de software.

A utilização do Git e do [GitHub](#) contribuiu para um melhor gerenciamento do projeto, permitindo controle das alterações, histórico de commits e organização das atividades através do Kanban.

Além disso, a implementação do CRUD e do GitHub Actions possibilitou aplicar conceitos técnicos importantes relacionados ao desenvolvimento web e automação de processos.

Por fim, o projeto demonstrou como as metodologias ágeis podem tornar o desenvolvimento mais organizado, flexível e eficiente, contribuindo para a entrega de um software mais confiável e bem estruturado.